LAB 1: Logistic Regression

OBJECTIVE:

1. To understand and apply the Machine Learning (ML) pipeline using Logistic Regression for a binary classification problem.

THEORY

Logistic Regression

Logistic regression is a supervised learning classification algorithm used to predict the probability of a target variable. The nature of target or dependent variable is dichotomous, which means there would be only two possible classes.

In simple words, the dependent variable is binary in nature having data coded as either 1 (stands for success/yes) or 0 (stands for failure/no).

Mathematically, a logistic regression model predicts P(Y=1) as a function of X. It is one of the simplest ML algorithms that can be used for various classification problems such as spam detection, Diabetes prediction, cancer detection etc.

Artificial Intelligence(AI)

Artificial intelligence (AI) is technology that enables computers and machines to simulate human learning, comprehension, problem solving, decision making, creativity and autonomy.

Applications and devices equipped with AI can see and identify objects. They can understand and respond to human language. They can learn from new information and experience. They can make detailed recommendations to users and experts. They can act independently, replacing the need for human intelligence or intervention (a classic example being a self-driving car).

Machine Learning(ML)

Machine learning is the subset of artificial intelligence (AI) focused on algorithms that can “learn” the patterns of training data and, subsequently, make accurate inferences about new data. This pattern recognition ability enables machine learning models to make decisions or predictions without explicit, hard-coded instructions.

Deep Learning(DL)

A subset of machine learning, deep learning (DL) trains computers to perform human tasks like speech recognition, image identification, and prediction making. It improves the ability

to classify, recognize, detect, and describe using data input.There has been an increased interest in DL as artificial intelligence (AI) has risen in popularity.

Data Science

Data science is inherently interdisciplinary as it combines expertise from statistics, computer science, mathematics, and domain-specific knowledge. This makes it incredibly versatile, with applications spanning healthcare, finance, marketing, and even environmental research.

For instance, a data scientist in public health can analyze demographic data in order to make predictions about the spread of a disease. Similarly, in the business sector, data science guides personalized marketing strategies by working with data related to customer behavior.

IMPLEMENTATION:

Task 1: Logistic regression with single feature.

For this task, we wiil be following and documenting steps of Machine Learning(ML) pipeline

1. Data Retrieval and Collection
2. Data Cleaning
3. Feature Design
4. Algorithm Selection
5. Loss Function Selection
6. Model Learning (Training)
7. Model Evaluation

1. Data Retrival and collection

This dataset contains 271 records merged from publicly available heart disease datasets. It includes 14 features that are crucial for predicting heart attack and stroke risks, covering both medical and demographic factors.

In [41]:
```
import pandas as pd
import numpy as np

# Loading the dataset
df = pd.read_csv("Heart_Disease_Prediction.csv")

# Displaying shape and columns
print("Dataset Shape:", df.shape)
print("Column Names:", df.columns.tolist())
```

```
Dataset Shape: (270, 14)
Column Names: ['Age', 'Sex', 'Chest pain type', 'BP', 'Cholesterol', 'FBS over 120',
'EKG results', 'Max HR', 'Exercise angina', 'ST depression', 'Slope of ST', 'Number
of vessels fluro', 'Thallium', 'Heart Disease']
```

2. Data Cleaning

In [42]:
```
print(df.isnull().sum())

# 2. Handling invalid/zero cholesterol values (if any)
# Total cholesterol cannot physiologically be 0; replace 0 with the median value
df['Cholesterol'] = df['Cholesterol'].replace(0, df['Cholesterol'].median())

# 3. Ensuring the target variable is binary (0 or 1)
print("Unique target values:", df['Heart Disease'].unique())

# 4. Verifying data types
print(df.dtypes)

# Mapping the string target values to binary integers (1 and 0)
df['Heart Disease'] = df['Heart Disease'].map({'Presence': 1, 'Absence': 0})

# Separating features (X) and target variable (y)
X = df.drop(columns=['Heart Disease'])
y = df['Heart Disease']
```

```
Age                       0
Sex                       0
Chest pain type           0
BP                        0
Cholesterol               0
FBS over 120              0
EKG results               0
Max HR                    0
Exercise angina           0
ST depression             0
Slope of ST               0
Number of vessels fluro   0
Thallium                  0
Heart Disease             0
dtype: int64
Unique target values: ['Presence' 'Absence']
Age                         int64
Sex                         int64
Chest pain type             int64
BP                          int64
Cholesterol                 int64
FBS over 120                int64
EKG results                 int64
Max HR                      int64
Exercise angina             int64
ST depression             float64
Slope of ST                 int64
Number of vessels fluro     int64
Thallium                    int64
Heart Disease              object
dtype: object
```

3. Feature Design

In [43]:
```
# Selecting single feature and target
X = df[['Cholesterol']]
y = df['Heart Disease']
```

Cholestrol is relevant predictor because high levels of low-density lipoprotein (LDL) cholesterol lead to plaque buildup in the arterial walls (atherosclerosis). This restricts blood flow and significantly increases the clinical risk of coronary artery disease over time, making it a highly relevant logical predictor for a machine learning model.

4. Algorithm Selection

We will be choosing Logistic Regression for this task.

Logistic regression is suitable for binary classification because it is a linear classification model rather than a simple regression model. Instead of fitting a straight line that can predict infinitely large or small values, it passes the linear combination of inputs through the Sigmoid function which makes it suitable for binary classification problems. It is mathemqatically expressed as:

$$S(z) = \frac{1}{1 + e^{-z}}$$

5. Loss Function Selection

For our Loss function, we will be using Binary Cross-Entropy (Log Loss).

Binary Cross-Entropy (Log Loss):

It is a standard loss function commonly used in machine learning for evaluating in binary classificaion problem where the output is either 0 or 1. It works by measuring the performance of a classification model wherer output is probability value between 0 and 1. It punishes the model for false prediction aggressively using logarithms.

6. Model training

In [44]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Splitting the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, shuffle=T

# Initializing and training the model
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)

#learned parameters
```

```
print("Learned Coefficient (Weight):", model.coef_)
print("Learned Intercept (Bias):", model.intercept_)
```

```
Learned Coefficient (Weight): [[0.00351491]]
Learned Intercept (Bias): [-1.04698442]
```

After successfully splitting dataset into training set and testing set, this logistic regression model was initialized and trained. The model learned this parameters through an iterative trial and error process called gradient descent. During model.fit(), it started with random baseline guesses for the weight and bias, calculated data-driven probabilities using the Sigmoid function, and evaluated its mistakes using a Log Loss penalty function. it continuously adjusted the parameters to minimize mistakes. stopped updating once it found original values.

7. Model Evaluation

In [45]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


y_pred = model.predict(X_test)

# Calculating Evaluation Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# Printing Performance Report
print(" Model Evaluation Metrics ")
print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-Score:  {f1:.4f}")
print("\nConfusion Matrix")
print(cm)

print("\n Detailed Classification Report ")
print(classification_report(y_test, y_pred))
```

```
 Model Evaluation Metrics
Accuracy:  0.6111
Precision: 0.5000
Recall:    0.2381
F1-Score:  0.3226

Confusion Matrix
[[28  5]
 [16  5]]

 Detailed Classification Report
              precision    recall  f1-score   support

           0       0.64      0.85      0.73        33
           1       0.50      0.24      0.32        21

    accuracy                           0.61        54
   macro avg       0.57      0.54      0.52        54
weighted avg       0.58      0.61      0.57        54
```

After evaluation, the model achieved accuracy of 61.11%. However, the underlying metrices reveal a severe limitation in it's ability to detect medical conditions.

Task 2: Logistic Regression with Multiple Features

In task 2, we will be using multiple feature. we will repeat the steps of ml pipelining till feature design.

1. Data Retrival and collection

In [46]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Loading the dataset
df = pd.read_csv("Heart_Disease_Prediction.csv")

# Displaying shape and columns
print("Dataset Shape:", df.shape)
print("Column Names:", df.columns.tolist())
```

```
Dataset Shape: (270, 14)
Column Names: ['Age', 'Sex', 'Chest pain type', 'BP', 'Cholesterol', 'FBS over 120',
'EKG results', 'Max HR', 'Exercise angina', 'ST depression', 'Slope of ST', 'Number
of vessels fluro', 'Thallium', 'Heart Disease']
```

2. Data Cleaning

In [47]:
```
print(df.isnull().sum())
```

```
# 2. Handling invalid/zero cholesterol values (if any)
# Total cholesterol cannot physiologically be 0; replace 0 with the median value
df['Cholesterol'] = df['Cholesterol'].replace(0, df['Cholesterol'].median())

# 3. Ensuring the target variable is binary (0 or 1)
print("Unique target values:", df['Heart Disease'].unique())

# 4. Verifying data types
print(df.dtypes)

# Mapping the string target values to binary integers (1 and 0)
df['Heart Disease'] = df['Heart Disease'].map({'Presence': 1, 'Absence': 0})

# Separating features (X) and target variable (y)
X = df.drop(columns=['Heart Disease'])
y = df['Heart Disease']
```

```
Age                         0
Sex                         0
Chest pain type             0
BP                          0
Cholesterol                 0
FBS over 120                0
EKG results                 0
Max HR                      0
Exercise angina             0
ST depression               0
Slope of ST                 0
Number of vessels fluro     0
Thallium                    0
Heart Disease               0
dtype: int64
Unique target values: ['Presence' 'Absence']
Age                           int64
Sex                           int64
Chest pain type               int64
BP                            int64
Cholesterol                   int64
FBS over 120                  int64
EKG results                   int64
Max HR                        int64
Exercise angina               int64
ST depression               float64
Slope of ST                   int64
Number of vessels fluro       int64
Thallium                      int64
Heart Disease                object
dtype: object
```

3. Feature design

In [48]:
```
selected_features = [
    'Age','Sex','Chest Pain Type','Resting Blood Pressure','Cholesterol',
    'Fasting Blood Sugar','Resting ECG','Max Heart Rate','Exercise Induced Angina',
    'Oldpeak','ST Slope'
```

```
]

# normalizing names and show available columns for debugging
selected_features = [s.strip() for s in selected_features]
print("DataFrame columns:", df.columns.tolist())
# reporting any missing names and select only existing columns
missing = [s for s in selected_features if s not in df.columns]
if missing:
    print("Missing features (will be ignored):", missing)

cols = [c for c in selected_features if c in df.columns]
if not cols:
    raise ValueError("No selected features found in dataframe columns. Update `sele

X = df[cols]
y = df['Heart Disease']
```

```
DataFrame columns: ['Age', 'Sex', 'Chest pain type', 'BP', 'Cholesterol', 'FBS over
120', 'EKG results', 'Max HR', 'Exercise angina', 'ST depression', 'Slope of ST', 'N
umber of vessels fluro', 'Thallium', 'Heart Disease']
Missing features (will be ignored): ['Chest Pain Type', 'Resting Blood Pressure', 'F
asting Blood Sugar', 'Resting ECG', 'Max Heart Rate', 'Exercise Induced Angina', 'Ol
dpeak', 'ST Slope']
```

Using multiple features improves prediction performance because it provides additionL complementary informatuin about outcome, increasing that model can detect true patterns. Also, cpmbination of features can interact to reveal relationships that single feature cannot.

4. Algorithm selection

we choose multi-variable Logistic regression. The linear combination shifts from a single variable slope ($z = \beta_0 + \beta_1 x$) to a multi-dimensional plane equation:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

The sigmoid transformation $S(z) = \frac{1}{1+e^{-z}}$ continues to map this multi-dimensional hyperplane onto a clean 0 to 1 probability spectrum.

5. Function Loss selection

We will use Binary cross Entropy(Log Loss). The math behaves exactly as it did in Task 1, but the optimization space expands to find the optimal global minima across all feature weights (β_1 through β_n) simultaneously.

6. Model Training

In [49]:
```
#Training the multi-feature model
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)
multi_model = LogisticRegression(random_state=42, max_iter=1000)
multi_model.fit(X_train, y_train)
```

```
print("Multi-Feature Model Training Complete.")
```

```
Multi-Feature Model Training Complete.
```

7. Model evaluation

In [50]:
```
# Predicting labels
y_pred_multi = multi_model.predict(X_test)

# Calculating Evaluation Metrics
accuracy_m = accuracy_score(y_test, y_pred_multi)
precision_m = precision_score(y_test, y_pred_multi)
recall_m = recall_score(y_test, y_pred_multi)
f1_m = f1_score(y_test, y_pred_multi)
cm_m = confusion_matrix(y_test, y_pred_multi)

print("Multi-Feature Model Evaluation Metrics")
print(f"Accuracy:  {accuracy_m:.4f}")
print(f"Precision: {precision_m:.4f}")
print(f"Recall:    {recall_m:.4f}")
print(f"F1-Score:  {f1_m:.4f}")
print("\nConfusion Matrix\n", cm_m)
```

```
Multi-Feature Model Evaluation Metrics
Accuracy:  0.6111
Precision: 0.6000
Recall:    0.4800
F1-Score:  0.5333

Confusion Matrix
 [[21  8]
 [13 12]]

Accuracy:  0.6111
Precision: 0.6000
Recall:    0.4800
F1-Score:  0.5333

Confusion Matrix
 [[21  8]
 [13 12]]
```

Discussion

In this lab, we implemented Logistic regression classifier for a supervised binary classification tasks. we followed the ML pipeline steps, we worked on a very useful dataset aboout heart disease. we used various python tools for initial step of data retrival and collection, we further cleaned the data to ensure no value is missing or invalid. we divided our task into two, single feature and multiple feature logistic regression.

For single feature, we selected cholestrol as only input feature and further separated features(X) and target(Y). Logistic regression was used as simple linear model. Binary cross

entropy also known as log loss was was our loss function. after successfully splitting dataset into training set and testing set, this logistic regression model was initialized and trained. our model was evaluated using four appropriate classification metrices, accuracy, precision, recall and F1-score. we also printed confusion matrix.

For multiple feature, we repeated most of steps like task 1(single feature). Most notable change was selection of feature,we used all available features and applied feature scaling where appropriate. we evaluated our model using same classification metrices as task 1.

one of main feature of our exercise was comparison between both task models. it provided us with some conclusive results about performance and capabilities of both model. Task 2 model was expected to perform better for its usage of more features and we founnd that task 2 model which used multiple features performed better, validating our hypothesis. since, multiple features usually capture different heart-disease signals, so the model can separate classes more reliably than using only one feature. More features usually improve accuracy by giving the model additional predictive signals and better class separation. it was the case in this exercise. Recall also had significant boost in task 2, reason for this is that when modeel can detect more true positives with additional relevant information, Recall improves too. However, when extra features are noisy, irrelevant and redundant, opposite can happen. Further we analyzed the trade-offs between interpretability and performance, the results we got were, fewer features yield simpler, more interpretable models (e.g., a single coefficient clearly showing cholesterol's effect, ideal for explanations and clinical use), while more features boost predictive performance by capturing complex patterns at the cost of interpretability. It can be balanced by choosing simple models when explainability matters most, richer models for accuracy, or techniques like feature selection, regularized regression, and coefficient analysis when both are needed.

CONCLUSION

To conclude, this Lab exercise divided in two task about training models and evaluating their performance was a successful one. we trained two models, one with single feature and other with multiple feature. we got an expected result, second model with multiple feature outperformed the single feature one in four appropriate classification metrices. some directions gained from our finding would be to use cross-validation and regularization to confirm gains and avoid overfitting. Also, if interpretability is required, prefer a single feature model over a dense multi-feature one.